故事线提纲（毕设）

1. 主题

作为毕设的题目：

基于 LLM 的 CISB dataset 构建及应用

2. 研究背景和现状

作为软件工程的基础设施，编译器产出的代码语义必须与源代码保持严格一致。因此编译器的正确性一直是备受社区关注的话题，需要充分的测试和验证。然而，代码语义上的等价并不能确保安全性。经过编译处理特别是编译优化后的代码，可能产生严重的安全问题，语义正确性和安全性之间存在差距，D'Silva 从形式上证明了这一点。

另一方面，编译器作为一种软件开发的产品，自身存在一定的使用规范需要开发者遵守，尽管这些规范可能鲜为人知。然而研究表明，程序员可能认为编译器产出的代码一定安全。因此，经过编译器处理后引入的漏洞就会成为 silent bug 长期潜伏在软件中，部分甚至长达十余年，造成严重的安全风险。

因此，产生这一类漏洞的原因可能是编译器、开发者、软硬件环境相关因而多方面的。先前的对编译器相关的漏洞研究不考虑以上因素的综合影响，则可能将这些编译器引入的安全漏洞简单视为 compiler bug 或 miscompilation。Xu 通过对 Bugzilla、Linux kernel 等编译器引入漏洞的报告的修补平台的人工调研，将这类漏洞命名为 CISB，提出一种三层分类架构，探明了 CISB 产生的根本原因和影响模式。

然而，现有的 CISB 研究方法——通过手动调研漏洞报告、代码补丁分析以及 CISB 筛选等安全任务均需要耗费大量人力，且高度依赖人类安全专家的知识，自动化程度和可重现性较低，限制了 CISB 作为一种特殊且危险的安全漏洞被广泛认识和研究的潜力。另一方面，传统漏洞方法往往只局限于源代码层面的分析，忽略了编译器、开发者、软硬件环境等多方面因素综合作用，对 CISB 这类语义更加复杂，触发条件更加苛刻的漏洞则难以适用。因此，本研究致力于提出一种自动化程度更高，易于重现的 CISB dataset 构建方案，并基于对 dataset 的研究提出一种 CISB 的检测方法。

近年来，LLM 凭借其丰富的领域知识和强大的代码语义理解能力，在安全研究尤其是漏洞检测中扮演越来越重要的角色。【列举一部分 LLM 直接作为/辅助安全任务的工作】。LLM 具备的编译器、语言规范知识，特定软硬件环境下安全需求等大量且冷僻的安全知识，而这些通常是开发者忽视或难以掌握的。部分研究表明，LLM 对数量庞大、篇幅冗长的漏洞报告和代码补丁的分析和总结能力，对代码模式的识别和提取能力远超人工，有助于提高对其审查的自动化，并进一步延伸于 CISB 的检测。因此，本研究尝试以 LLM 的安全知识辅助和代码分析能力部分替代人工审查，提出一种更高效的 CISB dataset 构建流水线；结合 LLM 的模式提取能力，提出一种基于模式匹配的 CISB 静态检测方法。

然而，当前部分研究也提出了对 LLM 直接用于安全任务有效性的质疑：①LLM 可能生成语法正确但语义错误的代码；②部分研究表明，LLM 的推理更依赖代码形式上的匹配而不是代码逻辑的推理；③部分对当前研究结果的评估表明，部分研究存在 benchmark 设置不合理，数据泄露等问题。基于以上考虑，在尝试建立自动化构建 CISB dataset 中，我们暂时认为 LLM 的输出不可信，不追求实现完全自动化的 bug report、patch history 审查；而是评估 LLM 直接处理构建过程的各阶段的性能和稳定性，识别其存在的局限性，在此过程中测量 LLM 的能力边界。将 LLM 作为一个知识丰富，反应迅速的 CISB 识别和筛选助手，总结与单纯人工审查的对比，突出其优越性和局限性。该评估过程仍然需要一定的人力。

经过以上评估，我们发现了直接利用 LLM 筛选和整理 CISB、提取 CISB 漏洞模式验证其

有效性的诸多挑战，根据评估结果结合提示词工程技术尝试解决这些挑战。基于评估建立的 LLM 工作流程，我们扩展了 Xu 的 CISB dataset 和分类框架；基于提取出的漏洞模式，我们在真实世界开源软件中找到了在野的 CISB。

3. 研究内容

基于以上内容，我们围绕自动化构建 CISB dataset 及检测应用这个总体目标，将其拆分为两个子模块：

一．自动化 CISB dataset 构建方法

- **i. 评估 LLM 直接构建 dataset 的各阶段**
- **ii. 基于评估结果建立 LLM 工作流程**

二．基于漏洞模式的静态检测

- **i. 从人工编写的 test case 中提取漏洞模式**
- **ii. 基于漏洞模式的真实世界代码 CISB 检测**

3.1 自动化的 CISB dataset 构建方法。

首先是利用 LLM 直接处理构建数据集的各个步骤，如漏洞报告、代码补丁的审计，代码分析，漏洞检测等。分别测试其性能和稳定性进行评估，识别其存在的局限性（如对 CISB 知识理解不够全面、输出不可信等），总结其中的主要挑战。然后基于评估结果，结合 LLM 相关技术缓解发现的缺陷，最后设计并构建一个优化后的工作流程实现 CISB dataset 的自动化构建。

存在的挑战与解决思路。CISB 在漏洞报告和代码补丁尤其是前者中，常常作为一种潜藏的语义蕴含在开发者的预期与编译器假设的冲突中。而最终 CISB 的触发通常是因为其在该报告或补丁所处的软硬件环境下，编译器对代码做出的具体处理引入了安全风险。因此利用 LLM 识别报告、补丁中潜藏的 CISB 并不能用是或不是的简单逻辑判断。在方案设计中，我们将评估过程按照开发者预期、编译器行为和软硬件环境展开。从问题描述中提取预期，根据代码块和执行结果推理编译器假设以及行为。评估冲突的后果在该环境下的安全影响，确定 CISB 是否存在。另一方面，LLM 自身仍存在一些局限性如幻觉、输出不确定、窗口有限等问题，我们将通过提示词工程、引入外部领域知识、模仿多 agent 形式部分缓解这些问题。

3.2 基于漏洞模式的静态检测。

通过对整理得到的 CISB 进行研究，我们发现其通常包含三方面的语义：①开发者编写触发漏洞代码的预期；②编译器针对该情景的假设和具体处理；③漏洞最终触发的某种软硬件环境。其中开发者的预期往往体现为一段特定的代码模式；已知的编译器的假设和行为体现为较确定的代码处理方法；而漏洞最终出发的环境在大部分 CISB 中是通用的。进一步，对编译器而言，确定的代码模式通常决定了对应确定的处理方法。即只要确定代码中存在某一段特定的模式，则意味着此处可能在编译器的处理下引入安全风险，成为潜在的 CISB 触发点。因此，我们提出了一套自动化分析提取漏洞模式并检验其有效性的机制。将得到的漏洞模式表示为代码片段的查询，最终实现代码库级别的 CISB 静态检测。

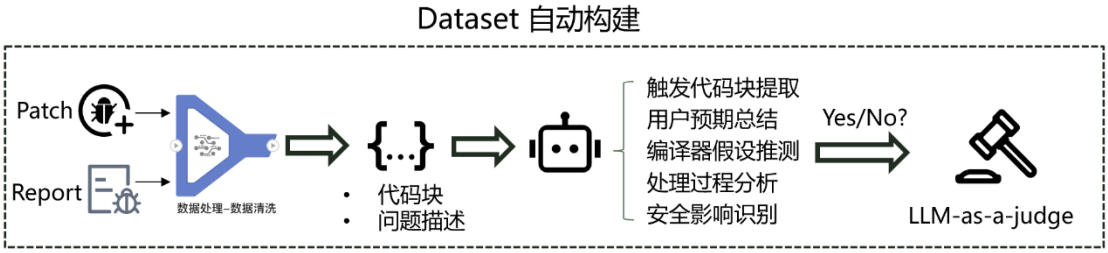
存在的挑战与解决思路。CISB dataset 中包含了不同类别 CISB 的对应测试程序以及说明，我们可以方便地从其中提取出代码模板和触发条件。这些测试程序大部分来自 CISB 发掘现场的程序，通常带有代码结构、CISB 语义和触发条件之外的冗余信息，导致 LLM 在提取时可能忽略部分关键模式、多提取非 CISB 语义的模式，得到的漏洞模式中存在噪声，降低生成的检测程序有效性。在方案设计中，我们需要保证输出中包含所有对应 CISB 语义的信息，并将测试程序中的说明总结归纳成对应的代码模板和触发条件，将模糊的自然语言转换为可

以用于明确判断的形式。

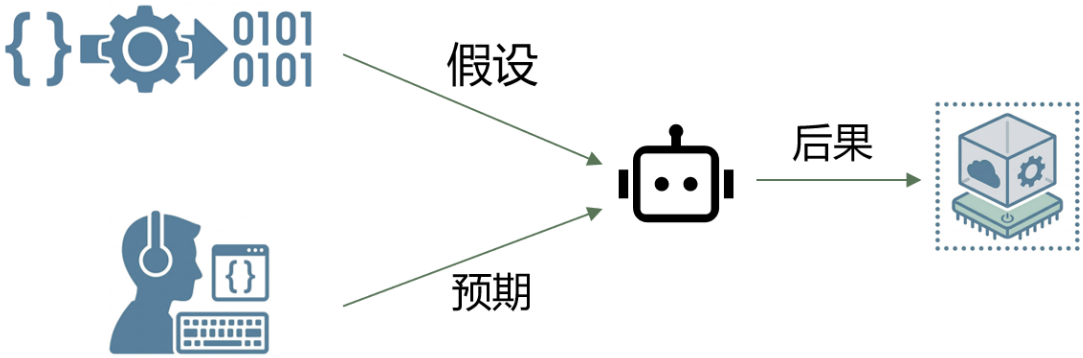
4. 研究方案

第一部分：

首先从 Bugzilla、Linux Kernel 等平台提取漏洞报告、代码补丁作为原始数据，使用关键词搜索进行定向筛选。通过初步的数据清洗，将原始数据结构化为代码块和问题描述两个部分输入给 LLM，依次完成以下任务：①根据问题描述，从清洗得到的代码块中定位 CISB 被触发的大致位置，提取其上下文；②从问题描述和上下文中总结出开发者编写代码的原始预期，检查是否有编程上的明显错误；③根据问题描述中运行产生的可观测结果反推编译器可能对原始代码做出的假设，判断是否与开发者的预期存在冲突；④分析编译器对原始代码进行的处理，以及⑤产生的后续安全影响。根据以上五个阶段的无人评估，综合判断是否存在潜藏的 CISB。



为缓解 LLM 自身的缺陷，我们结合提示词工程调教出了一套比较适用的模板。由于 LLM 的单次输出不确定性较高，我们将全过程拆分为信息提取和漏洞分析两个 agent，分别执行 ①② 和 ③-⑤ 阶段的任务，降低了单次输出不确定性的影响。为了解决通用模型领域知识缺乏、bugzilla 论坛上漏洞报告质量较低导致模型偏见和幻觉，进而使得 FP 和 FN 较高的问题，我们提取了 bugzilla 论坛上的开发者回复，作为外部的领域知识引入到提示词中。在最终裁判时，使用 checklist 综合判断 CISB 是否存在，从而避免单一判据对整体结果的决定性影响。实验证明，这些方法确实是有效的。

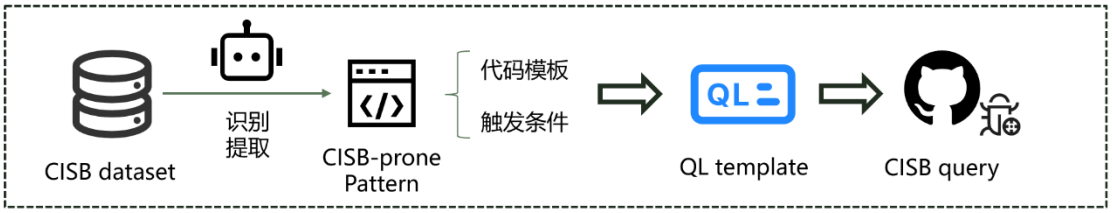


经过分析之后，我们的半自动化系统输出了一些疑似 CISB，还需要人类安全知识进行核对，才能最终证实为 CISB。之后，根据该漏洞触发过程中用户的预期与编译器假设的冲突，编译器对程序做出的破坏性处理，最后在该环境下引发的安全后果对 CISB 进行三个层次的分类，最终我们得到了一个可靠的、整理齐全的 ground truth 数据集。

第二部分：

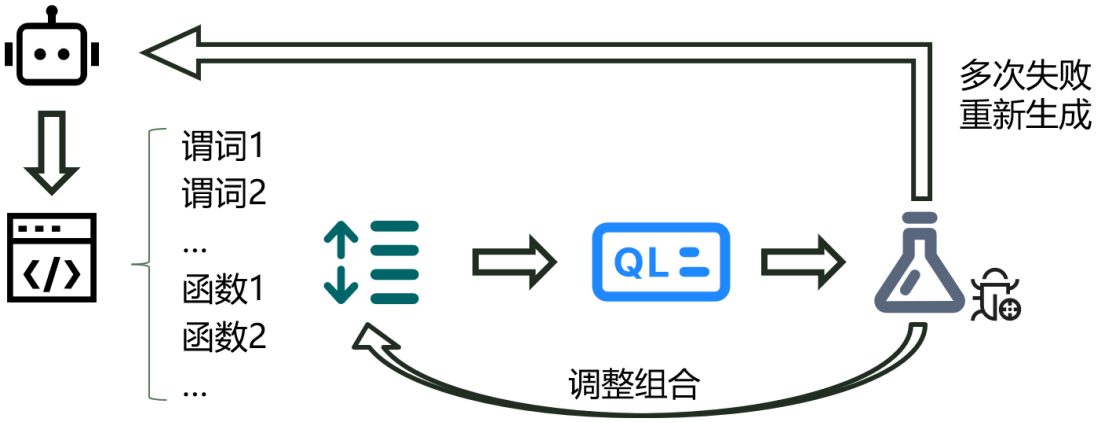
为了让 LLM 准确把握 CISB 语义和具体的触发代码之间的关系，首先要让 LLM 充分理解

dataset 中每一类 CISB 的测试程序。具体而言即上一部分中整理 CISB 的三层分类结构，根据程序中触发漏洞的代码片段推理，得到该类 CISB 的代码模板和触发的具体条件如编译选项、运行环境。之后将这些模式用 QL 语言编写为对应模板和条件的检测代码，查询 Github 上软件库的 CodeQL dataset，以实现真实世界软件的静态检测。



基于模式的检测

由于 LLM 对 CISB 的理解不一定准确，导致提取出的漏洞模式可能忽略关键的；或生成多余的、无关的模式或条件。为了进一步提高生成 QL 检测模板的有效性，我们设计了一套漏洞模式精炼机制。



首先，我们要求 LLM 将 CISB 的语义化约为 QL 语言的各种谓词和函数，以实现模块化的检测。初始时，将这些谓词和函数完整输入到 QL 模板中，根据对受测程序的检测结果有以下选择：①若未能检测到 CISB，则修改部分谓词或函数；②若检测到了 CISB，则尝试去除其中一个检测条件；③若多次失败，则要求 LLM 重新理解 CISB 后重新生成。经过精炼之后，我们可以大幅降低 FP 和 FN。

（这里的受测程序应该怎么设置没有想好，如果不能直接用 test case 能否用**程序合成？**）

5. 研究进展

到目前为止的进展：

- 调教出了一套比较好用的 CISB 推理提示词；
- 搭建好了完整的 CISB dataset 自动构建模型；
- 已经在漏洞报告和代码补丁的审计中发现了新 CISB；
- CISB 漏洞测试程序编写完成

理论成果：

- 自动化的 CISB dataset 构建方法
- 基于模式匹配的 CISB 检测方法

根据本课题的研究成果，将在在国际核心期刊上发表学术论文 1-2 篇。

时间安排：

- 2025 年 3 月— 9 月进行相关理论知识学习和调研文献；
- 2025 年 10 月— 12 月完善理论方案设计；
- 2026 年 1 月— 4 月进行系统开发与测试；
- 2026 年 5 月— 12 月进行期刊投稿；
- 2027 年 1 月— 4 月撰写毕业论文；
- 2027 年 4 月— 6 月准备答辩。